Chapter 2 — REST APIs: Principles, Design and Testing

Distributed Web Applications (AWD)

Dr. Badia Bouhdid
badiaa.bouhdid@esprit.tn

ESPRIT School of Engineering
4th-year Computer Engineering Program

Academic year 2026–2027

Learning outcomes

By the end of this chapter, students will be able to…

1 **LO 1** — Section 2

Explain the fundamental principles of the REST style

2 **LO 2** — Section 3

Identify business resources and expose them as a REST API

3 **LO 3** — Section 4

Distinguish the HTTP methods and their uses

4 **LO 4** — Section 5

Interpret the most common HTTP status codes

5 **LO 5** — Section 6

Understand the role of JSON in distributed applications

6 **LO 6** — Section 7

Understand why an API is documented with OpenAPI / Swagger

7 **LO 7** — Section 8

Test a REST API with an HTTP client

Technology-agnostic

Valid for any language or framework; implementation in the Spring Boot module.

Contents

1. Introduction
2. REST Principles
3. Resources and URI Design
4. HTTP Methods
5. HTTP Status Codes
6. JSON Representations
7. Documenting with OpenAPI
8. Testing a REST API
9. Summary

Section 1

Introduction

From Chapter 1 to Chapter 2

From *what* REST is to *how* to design, document and test a REST API

What Chapter 1 established

- Distributed applications communicate through **messages**.
- Web services expose an **interface contract**.
- REST: **resources** identified by URIs, manipulated with **HTTP verbs**, **stateless** server.

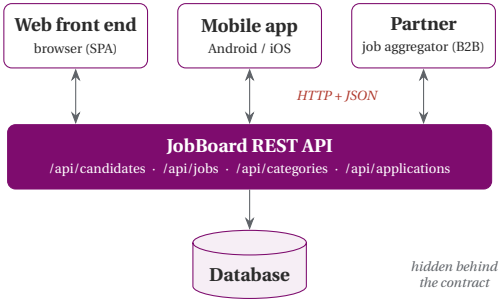
What Chapter 2 adds

- **Why** each REST constraint exists (Fielding).
- How to **design** resources and URIs.
- How to use **methods**, **status codes** and **JSON** correctly.
- How to **document** (OpenAPI) and **test** the API.

A REST API is a product: other teams build on it, so it must be predictable, documented and tested.

One back end, many clients

Why almost every modern application exposes a Web API

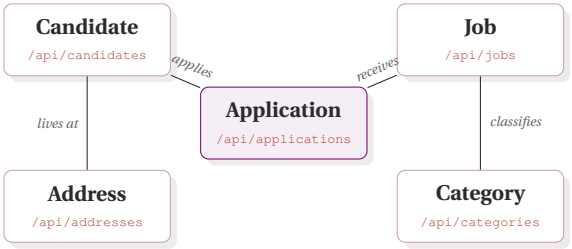


- One server, **several kinds of clients**, written in different languages.
- Clients only know the **contract**: URIs, methods, JSON formats, status codes.
- The server can change its code or database **without breaking** the clients, as long as the contract holds.

Decoupling: presentation and business logic evolve independently (*client–server* constraint).

Running example: the JobBoard platform

The same back end is used in the workshops and later split into microservices



Business rules (they will produce status codes)

- A candidate can apply to the same job **only once**.
- Only **available** jobs accept applications.
- A job or candidate with applications **cannot be deleted**.
- Candidate e-mails and category names are **unique**.
- The application date cannot be in the **future**.

Monolithic Spring Boot + MySQL back end, provided for Workshop 1. In this chapter we only look at its **API**, never at its code.

Section 2

REST Principles

REST: an architectural style, not a protocol

REpresentational State Transfer — Roy T. Fielding, PhD dissertation, UC Irvine (2000)

> *"REST provides a set of architectural constraints that, when applied as a whole, emphasizes scalability of component interactions, generality of interfaces, independent deployment of components, and intermediary components to reduce interaction latency, enforce security, and encapsulate legacy systems."*
>
> — R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures* (2000)

Architectural style

A named set of **constraints**. REST was used to guide the design of HTTP/1.1 and URIs.

Not a protocol

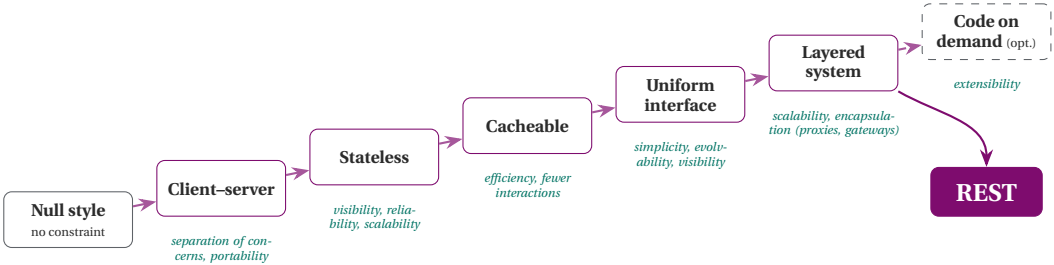
Unlike SOAP, REST defines no message format: it **reuses HTTP** as it was designed.

RESTful API

An API that respects the constraints: **resources**, **URIs**, **methods**, **representations**.

Deriving REST, one constraint at a time

Each constraint is added to obtain a property the Web needs



Adapted from R. T. Fielding (2000), Chapter 5, Figure 5-9 "REST derivation by style constraints".

Reading the figure: a constraint *restricts* what designers may do, and in exchange *guarantees* a property (in green). An API that drops a constraint loses the corresponding property.

Focus: the stateless constraint

Each request carries everything the server needs to process it

× Server-side session

```
// request 1
POST /login  {"email": "lina@..."}
-> Set-Cookie: SESSION=abc123
// request 2
GET /api/applications
Cookie: SESSION=abc123
-> the server must find the session
   in ITS memory
```

Problems: another instance does not know the session; a crash loses every session.

✓ Stateless: a token with every request

```
// request 1
POST /auth/token  {"email": "lina@..."}
-> {"token": "eyJhbGci..."}
// request 2 (self-contained)
GET /api/applications
Authorization: Bearer eyJhbGci...
-> the server checks the token,
   without remembering anything
```

Benefits: any instance can answer; add servers to scale out.

Stateless does not mean "without data": the database keeps the resources. Only the *conversation state* is not stored on the server.

Cacheable and layered system

Intermediaries can answer for the server — if the responses say so

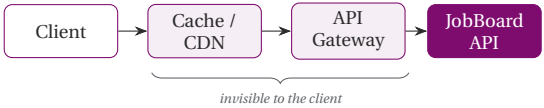
Conditional request with an `ETag`

```
GET /api/jobs/1
-> 200 OK
   Cache-Control: max-age=60
   ETag: "v7"
   {"id": 1, "name": "Java Developer", ...}

GET /api/jobs/1
If-None-Match: "v7"
-> 304 Not Modified      (no body)
```

The client reuses its copy: less bandwidth, faster answers.

Layered system



Each layer only sees the next one. Gateways add security, rate limiting or load balancing **without changing the client**.

The uniform interface

The central constraint of REST: four sub-constraints (Fielding, 2000, §5.1.5)

1. Identification

Every resource has a **URI**.

```
/api/jobs/1
```

2. Representations

The client acts on a resource through a **representation** it sends or receives.

```
PUT + JSON body
```

3. Self-descriptive

Each message says how to process it: **media type**, method, status code.

```
Content-Type:
application/json
```

4. Hypermedia

Responses contain **links** to the next possible actions (HATEOAS).

```
"_links": {...}
```

Same verbs, same codes, same media types for every resource: a client that knows HTTP already knows half of your API.

Hypermedia (HATEOAS) in practice

Hypermedia As The Engine Of Application State

```
GET /api/jobs/1   ->   200 OK
{
  "id": 1,
  "name": "Java Developer",
  "available": true,
  "_links": {
    "self":         { "href": "/api/jobs/1" },
    "category":     { "href": "/api/categories/1"
     },
    "applications": { "href": "/api/jobs/1/
     applications" },
    "apply":        { "href": "/api/applications",
                      "method": "POST" }
  }
}
```

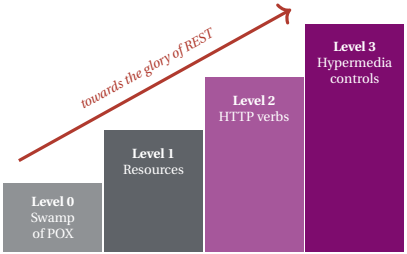
Adapted from the HAL convention (M. Kelly, IETF draft "JSON Hypertext Application Language").

- The client **follows links** instead of building URIs itself.
- The server can tell what is **allowed now**: no `apply` link if the job is closed.
- URIs can change without breaking clients.

In practice: most public "REST" APIs stop before hypermedia and rely on a documented contract (OpenAPI, Section 7).

The Richardson Maturity Model

How “RESTful” is an API? Four levels, from RPC tunnelling to hypermedia



Adapted from M. Fowler (2010), “Richardson Maturity Model: steps toward the glory of REST”, martinfowler.com — after L. Richardson (QCon 2008).

L0	One endpoint, everything in POST: `POST /api {"action":"getJob"}`
L1	One URI per resource, one verb: `POST /api/jobs/1/delete`
L2	Verbs + status codes: `DELETE /api/jobs/1` → `204`
L3	Responses carry **links** (HATEOAS).

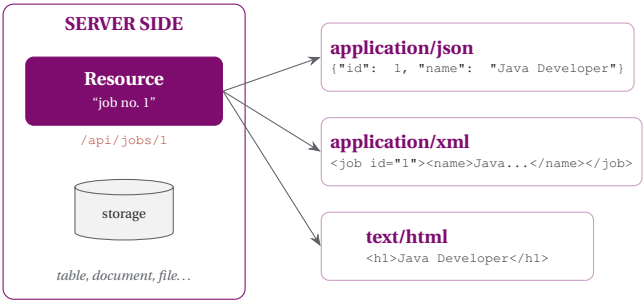
JobBoard is level 2, like most production APIs. Fielding considers level 3 a precondition for calling an API “REST”.

Section 3

Resources and URI Design

Resource and representation

The server manages resources; the client only ever receives representations



- A **resource** is any information that can be **named**: a job, a candidate, the list of jobs, today’s statistics.
- It is identified by a **URI** and stored as the server wishes — this is hidden from the client.
- A **representation** is a snapshot of the resource’s **state** in a given format.

REST = **RE**presentational **S**tate **T**ransfer: client and server exchange *representations of state*, never the resource itself.

Identifying the resources of a business application

Start from the nouns of the domain, not from the screens or the actions

A four-step method

1. List the **nouns** of the domain.
2. Keep those with an **identity** and a life cycle.
3. Each one becomes a **collection** and its **items**.
4. **Associations** become sub-resources or links.

"Job seekers apply to job offers published in categories; recruiters review the applications."

Noun	Collection	**Notes**
candidate	`/api/candidates`	identity: id, unique e-mail
address	`/api/addresses`	owned by a candidate
job (offer)	`/api/jobs`	has an *available* state
category	`/api/categories`	classifies jobs
application	`/api/applications`	links a candidate to a job
Associations → sub-resources		
job → applications	`/api/jobs/{id}/applications`	
candidate → applications	`/api/candidates/{id}/applications`	
category → jobs	`/api/categories/{id}/jobs`	

Tip: "apply" is a verb, but the *application* it creates is a noun: model the outcome of the action as a resource.

Resource archetypes

Four recurring kinds of resources

Document

A single item, with fields and links.

```
/api/jobs/1
/api/candidates/4
```

Collection

A server-managed set of items. The **server** chooses the new id (POST).

```
/api/jobs
```

Store

A client-managed set: the **client** chooses the URI (PUT).

```
/api/candidates/4/
favorites/12
```

Controller

A procedural action that fits no CRUD method. **Use sparingly.**

```
POST
/api/jobs/1/duplicate
```

Adapted from M. Massé, REST API Design Rulebook, O'Reilly (2011), Chapter 2.

Before inventing a controller, ask: can the action be expressed as creating or updating a resource?

Anatomy of a URI

Generic syntax defined by RFC 3986



Path = *which* resource, hierarchical from general to specific.

Query = *which view* of the resource: subset, order, page.

Fragment is never sent to the server.

Source: T. Berners-Lee, R. Fielding, L. Masinter, RFC 3986 — URI: Generic Syntax, IETF (2005).

URI design: good practices

The URI names a thing (noun); the HTTP method carries the action (verb)

✓ Do	× Avoid	Rule
`GET /api/jobs`	`GET /api/getAllJobs`	No verb in the URI
`POST /api/jobs`	`POST /api/createJob`	The action is carried by the method
`GET /api/jobs/1/applications`	`GET /api/applicationsOfJob?id=1`	Express the parent / child hierarchy
`GET /api/jobs?available=true`	`GET /api/jobs/available/true`	Filters, sorting, paging go in the query string
`/api/categories`	`/api/Category_List`	Plural nouns, lowercase, hyphens (not _)
`/api/job-offers`	`/api/jobOffers`	Hyphens improve readability
`/api/v1/jobs`	`breaking the contract silently`	Version the API on breaking changes
`/api/jobs/1`	`/api/jobs/1.json`	Format via `Accept`, not the extension

Source: M. Massé (2011); Microsoft, Web API design best practices, Azure Architecture Center; Zalando, RESTful API Guidelines.

Filtering, sorting, pagination and versioning

Same collection, different views — all in the query string

Filtering and sorting

```
GET /api/jobs?available=true&categoryId=2
GET /api/jobs?sort=date,desc
GET /api/candidates?lastname=Mansour
```

Pagination

```
GET /api/jobs?page=2&size=20
-> 200 OK
{ "content": [ {...}, {...} ],
  "page": 2, "size": 20,
  "totalElements": 134, "totalPages": 7 }
```

Versioning: three common options

URI path	`/api/v1/jobs` — simple, visible, cache-friendly
Header	`API-Version: 2` — clean URIs, less visible
Media type	`Accept: application/vnd.jobboard.v2+json`

Only breaking changes need a new version: removing a field, renaming it, changing its type. Adding an optional field is backward compatible.

Never

Return a 10 000-item collection in a single response: always paginate large collections.

Exposing JobBoard as a REST API

The endpoint map obtained by applying the method

Resource URI	GET	POST	PUT / PATCH	DELETE
`/api/candidates`	list	create	—	—
`/api/candidates/{id}`	read	—	replace	delete *(409 if it has applications)*
`/api/jobs`	list (filters)	create	—	—
`/api/jobs/{id}`	read	—	replace / update	delete *(409 if it has applications)*
`/api/categories, /{id}`	list, read	create	replace	delete
`/api/applications`	list (filters)	**apply**	—	—
`/api/applications/{id}`	read	—	update motivation	withdraw
`/api/jobs/{id}/applications`	applications received	*read-only sub-resource*		
`/api/candidates/{id}/applications`	applications sent	*read-only sub-resource*		
`/api/categories/{id}/jobs`	jobs of a category	*read-only sub-resource*		

Five resources, about 30 operations, a single pattern: once the rule is known, the whole API is predictable.

Section 4

HTTP Methods

Anatomy of an HTTP message

JobBoard example: a candidate applies to a job

Request (client → server)

```
POST /api/applications HTTP/1.1
Host: api.jobboard.tn
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGci...
(empty line)
{ "candidateId": 4, "jobId": 1,
  "motivation": "Join a backend team" }
```

start line

headers

body

Response (server → client)

```
HTTP/1.1 201 Created
Location: /api/applications/6
Content-Type: application/json
Date: Tue, 06 Oct 2026 09:12:00 GMT
(empty line)
{ "id": 6, "candidateId": 4, "jobId": 1,
  "applicationDate": "2026-10-06" }
```

Adapted from MDN Web Docs, "HTTP messages", Mozilla (developer.mozilla.org).

Start line: method + URI + version, or version + status code + reason.

Headers: metadata — format, authentication, caching, location.

Body: the representation (POST, PUT, PATCH, and most responses).

HTTP methods and CRUD operations

Semantics defined by RFC 9110 — *HTTP Semantics* (2022)

Method	CRUD	JobBoard example	Body	Safe	Idempotent	Typical success
GET	Read	`GET /api/jobs/1`	no	**yes**	**yes**	200 OK
POST	Create	`POST /api/jobs`	yes	no	no	201 Created + Location
PUT	Update (replace)	`PUT /api/jobs/1`	yes	no	**yes**	200 OK / 204
PATCH	Update (partial)	`PATCH /api/jobs/1`	yes	no	not guaranteed	200 OK / 204
DELETE	Delete	`DELETE /api/jobs/1`	no	no	**yes**	204 No Content
HEAD	—	`HEAD /api/jobs/1`	no	**yes**	**yes**	200, headers only
OPTIONS	—	`OPTIONS /api/jobs`	no	**yes**	**yes**	204 + `Allow` (CORS)

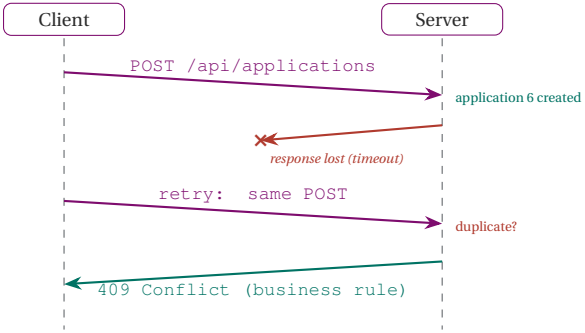
Safe: read-only — the client does not request any change of server state (GET, HEAD, OPTIONS).

Idempotent: sending the same request N times leaves the server in the same state as sending it once.

Source: IETF RFC 9110, §9.2 "Common method properties"; RFC 5789 for PATCH.

Why idempotence matters: retries

On a network, a lost response looks exactly like a lost request



- Clients, proxies and gateways **retry automatically** after a timeout.
- `GET`, `PUT`, `DELETE` can be retried **safely**: the final state is the same.
- `POST` is **not** idempotent: a retry may create a second application.

Protecting POST: a business rule (*one application per candidate and job* $\Rightarrow$ 409) or an `Idempotency-Key` header (IETF draft).

DELETE is idempotent even if the 2nd call returns 404: the server state no longer changes.

PUT or PATCH?

Replace the whole resource, or modify only part of it

PUT — full replacement

```
PUT /api/jobs/1 HTTP/1.1
Content-Type: application/json

{ "name": "Java Developer",
  "description": "REST API and microservices"
     ,
  "available": true,
  "date": "2026-09-15",
  "categoryId": 1 }
```

The client sends the **complete** representation. An omitted field is **lost** (reset to empty).

PATCH — partial update

```
PATCH /api/jobs/1 HTTP/1.1
Content-Type: application/merge-patch+json

{ "available": false }

-> 200 OK
{ "id": 1, "name": "Java Developer",
  "available": false, ... }
```

The client sends **only what changes**; the other fields stay unchanged.

Source: RFC 5789 (PATCH method), RFC 7396 (JSON Merge Patch), RFC 6902 (JSON Patch).

Creating a resource: POST or PUT?

Who chooses the identifier decides the method

POST on the collection

```
POST /api/applications
→ 201 Created
Location:  /api/applications/6
```

The **server** assigns the id. This is the usual case (database sequence, UUID).

PUT on the item

```
PUT /api/categories/data-science
→ 201 Created
```
(or `200` if it already existed)

The **client** knows the URI in advance (natural key, slug). Idempotent "create or replace".

Rule of thumb: POST to a *collection* creates a *new* member; PUT targets a *known* URI.

Wrong method? The server answers `405 Method Not Allowed` with an `Allow` header listing the accepted methods.

Section 5

HTTP Status Codes

Five classes of status codes

The first digit gives the class; the code is part of the API contract

1xx
Informational

Request received, continuing. Rare in JSON APIs.

2xx
Success

The request was received, understood and accepted.

3xx
Redirection

Further action needed: moved resource, `304` for caching.

4xx
Client error

The request is wrong: the **client** must change it.

5xx
Server error

A valid request the **server** failed to fulfil.

Why it matters: clients branch on the code, not on the message text. A wrong code breaks retries, caches and monitoring.

Monitoring: a rise in 5xx means a server problem; a rise in 4xx often means a badly written client.

Source: IETF RFC 9110, Section 15 “Status Codes”.

The codes you will use every day

With their meaning in the JobBoard API

2xx — Success

200 OK
`GET /api/jobs`

201 Created
`POST /api/applications` + `Location`

204 No Content
`DELETE /api/applications/6`

4xx — Client error

400 Bad Request — malformed JSON, invalid e-mail

401 Unauthorized — missing or expired token

403 Forbidden — authenticated but not allowed

404 Not Found — `GET /api/jobs/999`

405 Method Not Allowed — `POST /api/jobs/1`

409 Conflict — already applied, job closed

415 Unsupported Media Type — body sent as XML

5xx — Server error

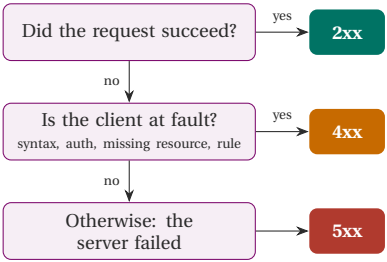
500 Internal Server Error
unhandled exception, bug

502 Bad Gateway
the gateway got an invalid answer from the service

503 Service Unavailable
database or dependency down, maintenance (+ `Retry-After`)

Choosing the right code

Three questions, then the most precise code



JobBoard scenario	Code
List the jobs	200 OK
Apply successfully	201 Created
Withdraw an application	204 No Content
Malformed JSON, invalid e-mail	400 Bad Request
Missing or expired token	401 Unauthorized
A candidate tries to publish a job	403 Forbidden
`GET /api/jobs/999`	404 Not Found
Apply twice to the same job	409 Conflict
Database unreachable	500 / 503

Classic mistakes

Each one makes the API harder to use correctly

200 OK with an error inside

`200 OK {"error": "job not found"}` — the client can no longer trust the code. **Use 404.**

Confusing 401 and 403

401: “who are you?” (not authenticated). 403: “I know who you are, but you are not allowed”.

Confusing 400 and 409

400: the request itself is invalid. 409: the request is valid but **conflicts with the current state** (already applied).

500 for a client error

A missing field or a wrong id is not a server failure. 5xx must mean “our fault”.

Be precise but not exotic: a dozen well-understood codes cover almost every API.

Standardising errors: Problem Details

RFC 9457 (2023, obsoletes RFC 7807) — one error format for every API

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "Candidate 4 already applied to job 1",
  "instance": "/api/applications"
}
```

`type`	URI identifying the error type (`about:blank` if the code is enough)
`title`	short, human-readable summary of the type
`status`	the HTTP code, repeated for logs
`detail`	explanation of *this* occurrence
`instance`	URI concerned by the error
extensions	e.g. `"errors": [{"field": "email", ...}]` for a 400

Security rule

Never a stack trace or an SQL message in a response: a clear message for the client, the technical details in the server logs.

Every client handles errors the same way, whatever the API. Most frameworks produce this format natively.

Source: M. Nottingham, E. Wilde, S. Dalal, RFC 9457 — Problem Details for HTTP APIs, IETF (2023).

Section 6

JSON Representations

Why an exchange format?

Distributed components written in different languages must agree on a neutral format



Interoperability

Any language can produce and parse it: no shared code, no shared runtime.

Decoupling

The internal model (classes, tables) can change; the representation stays stable.

Self-description

The media type (`application/json`) tells the receiver how to read the body.

JSON: JavaScript Object Notation

RFC 8259 / ECMA-404 — lightweight, readable, language-independent

```
GET /api/jobs/1  ->  200 OK
{
  "id": 1,
  "name": "Java Developer",
  "available": true,
  "date": "2026-09-15",
  "category": {
    "id": 1,
    "name": "Software Development"
  },
  "requirements": ["REST", "SQL"],
  "salary": null
}
```

Six value types

Type	Syntax	In the example
Object	`{ "k": v }`	`"category"`
Array	`[ v, v ]`	`"requirements"`
String	`"..."`	`"name"`, `"date"`
Number	`1, 2800.5`	`"id"`
Boolean	`true / false`	`"available"`
Null	`null`	`"salary"`

No date type: dates travel as ISO 8601 strings (`"2026-09-15"`, `"2026-09-15T09:12:00Z"`). No comments, no trailing commas.

Source: T. Bray (ed.), RFC 8259 — The JSON Data Interchange Format, IETF (2017); ECMA-404 (2017).

JSON or XML?

The same job, two representations

```json
{
  "id": 1,
  "name": "Java Developer",
  "available": true,
  "requirements": ["REST", "SQL"]
}
```

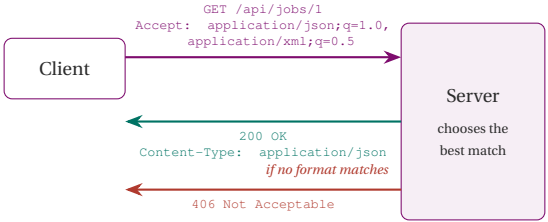
```xml
<job id="1">
  <name>Java Developer</name>
  <available>true</available>
  <requirements>
    <requirement>REST</requirement>
    <requirement>SQL</requirement>
  </requirements>
</job>
```

	JSON	**XML**
Verbosity	compact	tags repeated, about 2× larger
Data types	number, boolean, null, array	everything is text (unless a schema)
Browser support	native (`JSON.parse`)	DOM parser
Schema	JSON Schema (optional)	XSD, very rich
Typical use	REST APIs, configuration, NoSQL	SOAP, documents, legacy B2B

REST does not impose JSON: it imposes *representations*. JSON became the default because it maps directly to the data structures of most languages.

Content negotiation

The client says what it accepts; the server says what it sends



Adapted from MDN Web Docs, "Content negotiation", Mozilla (developer.mozilla.org).

`Accept`	format(s) the client wants in the response
`Content-Type`	format of the body actually sent
`406`	no acceptable representation
`415`	request body in an unsupported format

Content-Type appears in **both** directions: on a request it describes the body sent by the client (POST, PUT, PATCH).

Designing good representations

A representation is a public view, not a copy of the database row

× Database row exposed as is

```
{ "CAND_ID": 4,
  "first_name": "Lina",
  "pwd_hash": "$2a$10$Xy...",
  "address_id": 7,
  "created": 1759740720 }
```

Internal names, a secret leaked, a timestamp nobody can read, a bare foreign key.

✓ Designed for the client

```
{ "id": 4,
  "firstname": "Lina",
  "lastname": "Mansour",
  "email": "lina.mansour@example.com",
  "address": { "street": "...", "zipCode": "
     2080" },
  "createdAt": "2025-10-06T09:12:00Z" }
```

Consistent camelCase, ISO 8601 dates, no secrets, related data embedded or linked.

Separate the API model (DTO) from the persistence model: the database can change without breaking the contract.

Section 7

Documenting with OpenAPI

Why is Postman not enough?

An excellent test client… but not an API contract

What Postman does well

- Send any HTTP request
- Organise requests into collections
- Manage environments (local, test, prod)
- Write verification scripts

Its limits as documentation

Not a contract	shows what someone tested, not what the API guarantees
Maintained by hand	drifts as soon as the API changes
No formal schema	required fields, types, possible errors stay implicit
Hard to discover	a new developer or partner does not know what exists
Not machine-readable	no code generation, no mock, no validation

We need a single source of truth: a standard, machine-readable description of the API. Postman can import it — the two complement each other.

OpenAPI and Swagger: what is what?

OpenAPI is the *what* (the standard); Swagger is the *how* (the tools)

OpenAPI Specification (OAS)

A **vendor-neutral standard** to describe HTTP APIs in YAML or JSON, maintained by the **OpenAPI Initiative** (Linux Foundation).

Swagger

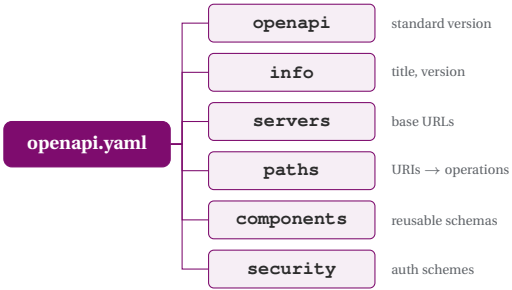
A **family of tools** built around the standard (SmartBear): Swagger UI, Swagger Editor, Swagger Codegen.



Source: OpenAPI Initiative, OpenAPI Specification v3.1.0 (2021), spec.openapis.org; SmartBear, swagger.io.

Structure of an OpenAPI document

Everything a client needs, in one file



- `paths` contains everything taught so far: **URIs**, **methods**, **parameters**, **status codes**, **representations**.
- `components/schemas` defines each model (`Job`, `Candidate`…) **once**, reused with `$ref`.
- The file is versioned with the code, like any other source file.

Adapted from OpenAPI Initiative, OpenAPI Specification, "OpenAPI Object" (spec.openapis.org).

Reading an OpenAPI excerpt

JobBoard's `GET /api/jobs/{id}` endpoint, in YAML

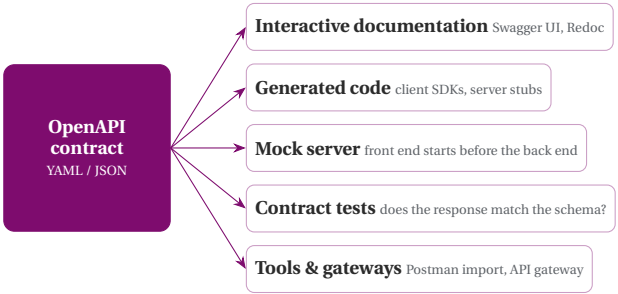
```yaml
openapi: 3.0.3
info:
  title: JobBoard API
  version: 1.0.0
paths:
  /api/jobs/{id}:
    get:
      summary: Get a job by its identifier
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        '200':
          description: Job found
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Job'
      }
        '404':
          description: Job not found
```

1. `paths`: the **resource** URI
2. `get`: the **method**
3. `parameters`: path, query or header, typed, required or not
4. `responses`: every possible **status code**
5. `schema`: the **JSON representation**

Documenting the 404 matters as much as the 200: errors are part of the contract.

The contract as a source of truth

Design first or code first — the contract feeds the whole tool chain



Design first

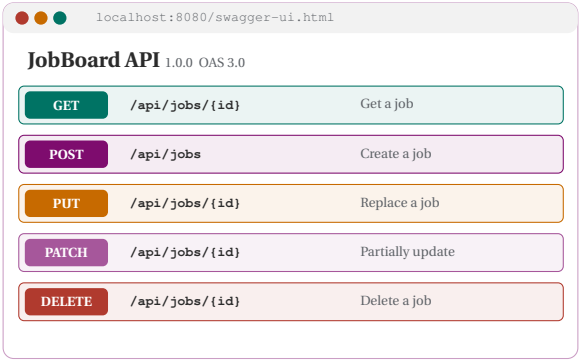
The contract is written **before** the code, reviewed by front, mobile and back teams, then implemented. Teams work in parallel.

Code first

A framework library **generates** the contract from the code and its annotations (shown in the Spring Boot module).

Swagger UI: documentation you can test

Generated from the contract: the documentation never drifts from the API



Testing an endpoint

1. Expand the endpoint: parameters, schemas, responses
2. **Try it out** makes the fields editable
3. Enter parameters and the JSON body
4. **Execute**: the curl command, the code and the response are displayed

Warning

Swagger UI really calls the API: a DELETE really deletes the data.

Section 8

Testing a REST API

What should an API test check?

The whole contract — not only the happy path

1. Status code

`201`, not `200`, after a POST; `404` for a missing resource.

2. Headers

`Content-Type`, `Location` after a creation, caching headers.

3. Body

Fields present, types and formats conform to the schema.

4. Error cases

400, 404, 409… and their Problem Details body. **Most often forgotten.**

5. Side effects

Was the resource really created, updated, deleted? Read it back.

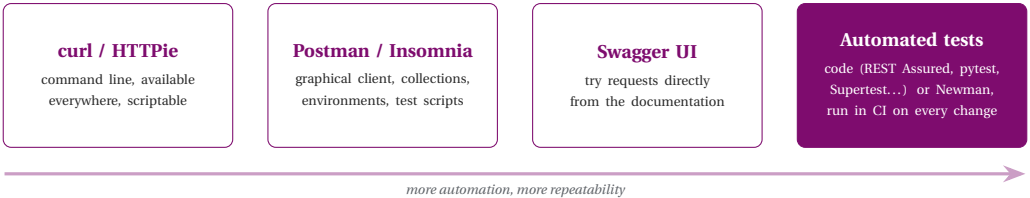
6. Security & performance

401 without a token, 403 without rights, acceptable response time.

A test is: a request, an expected code, expected headers, an expected body — and a check of the server state.

HTTP clients and testing tools

From manual exploration to automated regression tests



Explore by hand to understand an API and reproduce a bug: curl, Postman, Swagger UI.

Automate what must never break: tests replayed on every commit by the continuous integration pipeline.

curl in practice

The universal HTTP client — `-i` shows the response headers

```
# Read a collection, with a filter
curl -i "http://localhost:8080/api/jobs?available=true"

# Create a resource
curl -i -X POST http://localhost:8080/api/applications \
     -H "Content-Type: application/json" \
     -d '{"candidateId": 4, "jobId": 1}'

HTTP/1.1 201 Created
Location: /api/applications/6
Content-Type: application/json
{"id":6,"candidateId":4,"jobId":1,...}

# Delete it
curl -i -X DELETE http://localhost:8080/api/applications/6
HTTP/1.1 204 No Content
```

`-X`	HTTP method (GET by default)
`-H`	add a header
`-d`	request body
`-i`	include the response headers
`-v`	verbose: full exchange

Swagger UI shows the equivalent **curl command** for each request it executes: a quick way to learn.

Designing test cases for an endpoint

`POST /api/applications` — one nominal case is not enough

#	Situation	Request body	Expected
1	Valid application to an available job	`{"candidateId":4,"jobId":1}`	**201** + Location
2	Same request a second time	`{"candidateId":4,"jobId":1}`	**409** Conflict
3	Job closed (`available=false`)	`{"candidateId":4,"jobId":4}`	**409** Conflict
4	Unknown candidate	`{"candidateId":999,"jobId":1}`	**404** Not Found
5	Missing required field	`{"jobId":1}`	**400** Bad Request
6	Malformed JSON	`{"candidateId":4,`	**400** Bad Request
7	Application date in the future	`..."applicationDate":"2030-01-01"`	**400** Bad Request
8	Body sent as XML	`<application>...`	**415** Unsupported

Technique: one case per **business rule** and per **class of invalid input** (equivalence partitioning).

Independence: each test prepares its own data, so tests can run in any order.

Automating the checks

A test script asserts the contract instead of a human reading the response

Postman test script (JavaScript)

```
// POST /api/applications (2nd time)
pm.test("returns 409 Conflict", () =>
  pm.response.to.have.status(409));

pm.test("uses Problem Details", () =>
  pm.expect(pm.response.headers.get("Content-Type"))
    .to.include("application/problem+json"));

pm.test("explains the conflict", () =>
  pm.expect(pm.response.json().detail)
    .to.include("already applied"));
```

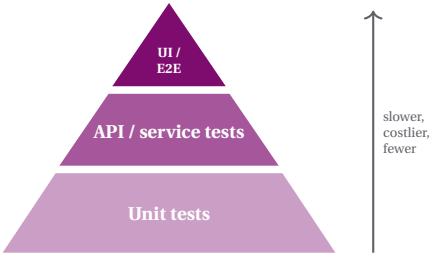
The same test, in any language

Given candidate 4 has applied to job 1
When `POST /api/applications` with the same ids
Then status is `409`
And `Content-Type` is `application/problem+json`
And `detail` mentions the duplicate

Run collections in CI with **Newman**, or write the same checks with the test library of your language.

Where API tests fit: the test pyramid

Many fast tests at the bottom, few slow end-to-end tests at the top



Adapted from M. Cohn, Succeeding with Agile, Addison-Wesley (2009); H. Vocke, "The Practical Test Pyramid", martinfowler.com (2018).

- **API tests** call the real HTTP interface: they check the **contract** without a browser.
- They are much faster and more stable than UI tests.
- **Contract tests** check that the responses match the OpenAPI schema — documentation and implementation cannot drift apart.

OpenAPI describes, tests verify: the contract says what must be true; the tests prove that it is.

Section 9

Summary

Key takeaways

One key idea per learning outcome

LO 1 — Principles

REST is a style: constraints that buy scalability, simplicity and evolvability. Stateless is key.

LO 2 — Resources

Nouns of the domain become collections and items; associations become sub-resources.

LO 3 — Methods

GET, POST, PUT, PATCH, DELETE ↔ CRUD, with their safety and idempotence.

LO 4 — Status codes

2xx / 4xx / 5xx are part of the contract; errors use Problem Details.

LO 5 — JSON

A neutral, typed text format: the representation, designed for clients, not a database dump.

LO 6 — OpenAPI

The contract is the source of truth; Swagger UI makes it readable and testable.

LO 7 — Testing

Check code, headers, body, errors and side effects — then automate.

A good REST API is predictable.

Same rules for every resource, and a contract that says so.

Test your knowledge
One question per learning outcome

LO 1 Why does the stateless constraint make a REST API easier to scale? What does it *not* forbid?

LO 2 Give the URIs to list the applications received by job 3, and the available jobs of category 2.

LO 3 Why is POST not idempotent? Is DELETE still idempotent if the second call returns 404?

LO 4 A candidate applies twice to the same job: 400 or 409? An authenticated candidate creates a job: 401 or 403?

LO 5 What is the difference between `Accept` and `Content-Type`? When does a server answer 406?

LO 6 Give three things an OpenAPI contract enables that a Postman collection does not.

LO 7 List the test cases you would write for `DELETE /api/jobs/{id}`.

Mini-exercise: design the request

In pairs, 5 minutes — method, URI and expected status code

1 List the available jobs of category 2
method · URI · code?

2 Create a new “Data” category
method · URI · code · header?

3 Close job offer no. 3
method · URI · body · code?

4 View the applications of candidate 4
method · URI · code?

5 Delete candidate 2, who has already applied
method · URI · code?

6 Apply a second time to the same job
method · URI · code · error format?

Next: Workshop 1 — run the JobBoard back end, read its contract in Swagger UI, and test it.

References

[1] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[2] L. Richardson, S. Ruby. *RESTful Web Services*. O'Reilly, 2007.

[3] M. Fowler. *Richardson Maturity Model: steps toward the glory of REST*. martinfowler.com, 2010.

[4] M. Massé. *REST API Design Rulebook*. O'Reilly, 2011.

[5] A. Lauret. *The Design of Web APIs*. Manning, 2019.

[6] R. Fielding, M. Nottingham, J. Reschke (eds.). *RFC 9110 — HTTP Semantics*. IETF, 2022.

[7] T. Berners-Lee, R. Fielding, L. Masinter. *RFC 3986 — Uniform Resource Identifier (URI): Generic Syntax*. IETF, 2005.

[8] L. Dusseault, J. Snell. *RFC 5789 — PATCH Method for HTTP*. IETF, 2010. P. Hoffman, J. Snell. *RFC 7396 — JSON Merge Patch*. IETF, 2014.

[9] T. Bray (ed.). *RFC 8259 — The JavaScript Object Notation (JSON) Data Interchange Format*. IETF, 2017.

[10] M. Nottingham, E. Wilde, S. Dalal. *RFC 9457 — Problem Details for HTTP APIs*. IETF, 2023.

[11] OpenAPI Initiative. *OpenAPI Specification v3.1.0*. spec.openapis.org, 2021.

[12] Mozilla. *MDN Web Docs — HTTP messages; Content negotiation*. developer.mozilla.org.

[13] M. Cohn. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley, 2009. H. Vocke. *The Practical Test Pyramid*. martinfowler.com, 2018.

[14] Microsoft. *Web API design best practices*. Azure Architecture Center. Zalando. *RESTful API and Event Guidelines*.

Thank you!

Questions? · Next: Workshop 1 — testing the JobBoard API